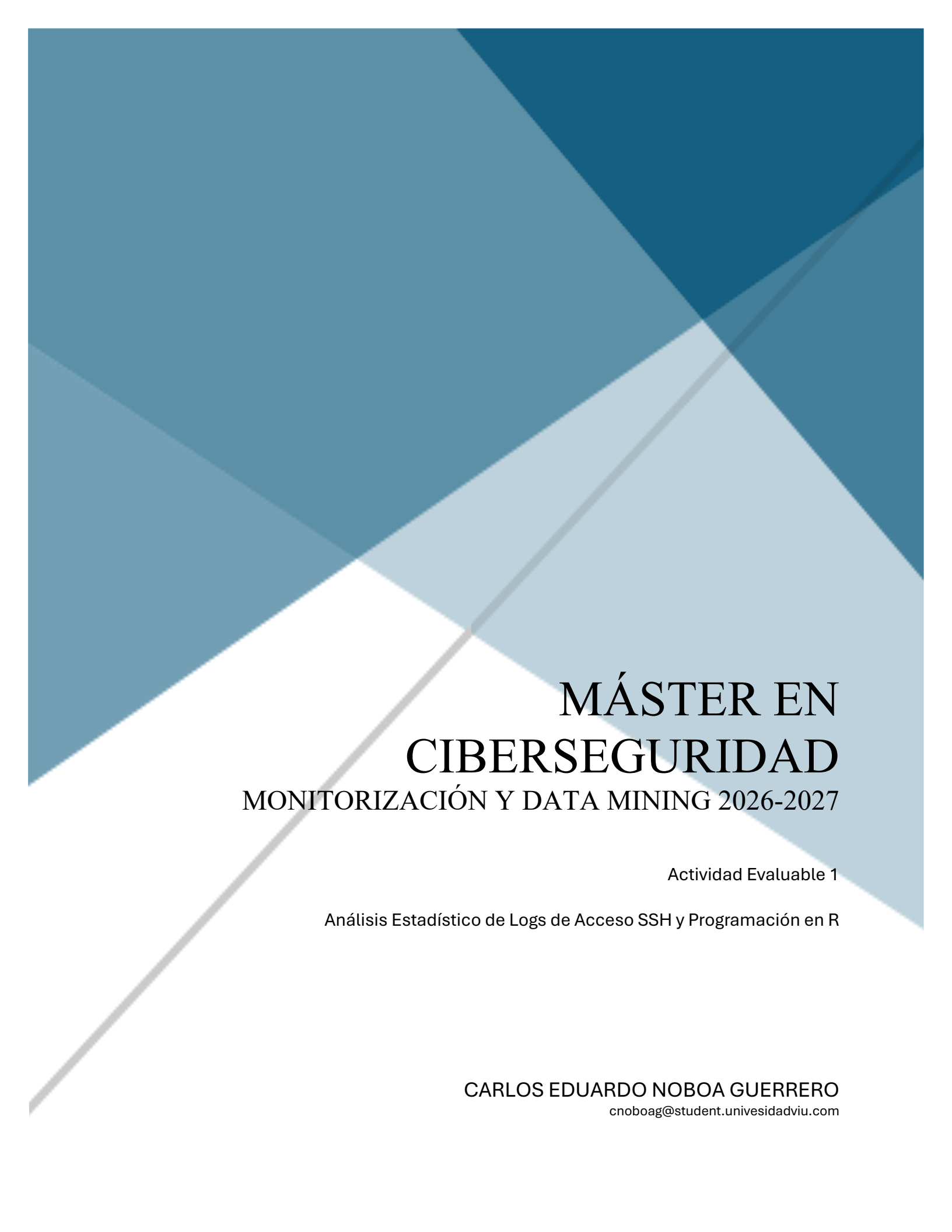




MÁSTER EN CIBERSEGURIDAD

MONITORIZACIÓN Y DATA MINING 2026-2027

Actividad Evaluable 1

Análisis Estadístico de Logs de Acceso SSH y Programación en R

CARLOS EDUARDO NOBOA GUERRERO
cnoboag@student.univesidadviu.com

Tabla de contenido.

INTRODUCCIÓN	2
OBJETIVO	2
DESARROLLO DE LA ACTIVIDAD	2
<i>PREGUNTA 1: Gestión de directorios y carga local del CSV.....</i>	<i>2</i>
<i>PREGUNTA 2: Creación de variables y comprobación de clases/tipos de datos</i>	<i>5</i>
<i>PREGUNTA 3: Operaciones estadísticas con el archivo .csv (Análisis de Logs)</i>	<i>7</i>
<i>PREGUNTA 4: Operaciones y Consultas sobre un Data.Frame.....</i>	<i>10</i>
<i>PREGUNTA 5: Operaciones Avanzadas con Listas.....</i>	<i>13</i>
<i>PREGUNTA 6: Funciones Personalizadas y Vectorización</i>	<i>16</i>
CONCLUSIONES.....	20

Tabla de imagenes.

<i>Imagen 1. Resultados en R para gestión de directorios y carga local del CSV</i>	<i>4</i>
<i>Imagen 2. Resultados en R para creación de variables y comprobación de clases/tipos de datos</i>	<i>6</i>
<i>Imagen 3. Resultados en R para operaciones estadísticas con el archivo .csv (Análisis de Logs).....</i>	<i>9</i>
<i>Imagen 4. Resultados en R para Operaciones y Consultas sobre un Data.Frame</i>	<i>12</i>
<i>Imagen 5. Resultados en R para Operaciones Avanzadas con Listas</i>	<i>15</i>
<i>Imagen 6. Resultados en R para Funciones Personalizadas y Vectorización</i>	<i>18</i>

Tablas de comandos.

<i>Tabla 1. Comandos para gestión de directorios y carga local del CSV.....</i>	<i>4</i>
<i>Tabla 2. Resultados en R para creación de variables y comprobación de clases/tipos de datos</i>	<i>7</i>
<i>Tabla 3. Comandos para operaciones estadísticas con el archivo .csv (Análisis de Logs)</i>	<i>9</i>
<i>Tabla 4. Comandos para operaciones y Consultas sobre un Data.Frame.....</i>	<i>12</i>
<i>Tabla 5. Comandos para Operaciones Avanzadas con Listas</i>	<i>15</i>
<i>Tabla 6. Comandos para Funciones Personalizadas y Vectorización</i>	<i>19</i>

Introducción.

El análisis de registros o *logs* de acceso es una tarea crítica en ciberseguridad para identificar vectores de ataque y garantizar la integridad de los servidores. El protocolo SSH, al ser una de las principales vías de administración remota, es constantemente blanco de intentos de acceso no autorizados. Esta práctica utiliza el lenguaje R para procesar de forma masiva el archivo `ssh_login_attempts-v2.csv`, permitiendo transformar datos crudos de red en información analítica estructurada para la toma de decisiones.

Para lograr un análisis eficiente, la práctica inicia estableciendo un entorno de trabajo reproducible, automatizando la gestión de directorios locales y la carga correcta del dataset en la memoria de RStudio. A través del desarrollo de este script, se demuestra cómo la correcta configuración inicial del entorno, sumada a técnicas de programación funcional y vectorización, optimiza la auditoría de seguridad y asienta las bases metodológicas del Data Mining.

Objetivo.

El objetivo principal de esta práctica es automatizar la configuración de un entorno de trabajo reproducible en RStudio para realizar la ingesta, manipulación y análisis estadístico descriptivo de un registro de accesos SSH (`ssh_login_attempts-v2.csv`). Mediante el desarrollo de un script estructurado, se busca dominar el control de directorios locales, la validación de tipos de datos, el filtrado eficiente de estructuras complejas (`Data.Frames` y listas) y la implementación de programación funcional vectorizada para optimizar auditorías de seguridad en Data Mining.

Desarrollo de la actividad

PREGUNTA 1: Gestión de directorios y carga local del CSV

A. Contexto y Planteamiento del Problema

En la fase inicial de todo proyecto de Data Mining, es crítico asegurar la disponibilidad y persistencia de las fuentes de datos. Dado que los enlaces de descarga web pueden quedar obsoletos o sufrir caídas de servicio, se ha optado por un enfoque de arquitectura local reproducible.

El script automatiza la preparación del espacio de almacenamiento en el sistema de archivos local (sincronizado con OneDrive), verifica de manera lógica si el directorio ya existe para evitar errores o sobrescrituras y realiza la ingesta del archivo de logs de seguridad `ssh_login_attempts-v2.csv` directamente en el entorno de memoria de RStudio.

B. Proceso de desarrollo de la solución

Paso 1. Se define una variable llamada `ruta_base` que contiene la dirección de la carpeta de documentos en OneDrive.

Código: `ruta_base <- "C:/Users/cenob/OneDrive/Documents"`

Paso 2. Se une la ruta base con el nombre de la carpeta `"data_mining"` usando una barra diagonal como separador.

Código: `data_mining <- paste(ruta_base, "data_mining", sep = "/")`

Paso 3. Se verifica si la carpeta `"data_mining"` ya existe; si no es así, se crea de forma automática.

Código: `if (!dir.exists(data_mining)) { dir.create(data_mining) }`

Paso 4. Se configura la carpeta `"data_mining"` como el directorio de trabajo activo en R.

Código: `setwd(data_mining)`

Paso 5. Se guarda el nombre del archivo CSV (`"ssh_login_attempts-v2.csv"`) en una variable local.

Código: `logs_practical <- "ssh_login_attempts-v2.csv"`

Paso 6. Se lee y carga el archivo CSV en la memoria del programa sin convertir los textos en factores.

Código: `dataset <- read.csv(logs_practical, stringsAsFactors = FALSE)`

Paso 7. Se muestran las primeras filas del dataset en la consola para confirmar que los datos se cargaron correctamente.

Código: `head(dataset)`

C. Código en Rstudio

1. # Paso 1: Definir la ruta base en tus Documentos de OneDrive
2. `ruta_base <- "C:/Users/cenob/OneDrive/Documents"`
3. # Paso 2: Crear la ruta de la carpeta uniendo la palabra 'data_mining'
4. `data_mining <- paste(ruta_base, "data_mining", sep = "/")`
5. # Paso 3: Comprobar si la carpeta ya existe; si no existe, R la creará automáticamente
6. `if (!dir.exists(data_mining)) {`
7. `dir.create(data_mining)`

8. }

9. # Paso 4: Establecer 'data_mining' como el directorio de trabajo activo

10. *setwd(data_mining)*

11. # Paso 5: Definir la variable con el nombre del archivo local

12. *logs_practical <- "ssh_login_attempts-v2.csv"*

13. # Paso 6: Cargar el archivo CSV en la memoria de RStudio

14. *dataset <- read.csv(logs_practical, stringsAsFactors = FALSE)*

15. # Paso 7: Comprobar la carga correcta visualizando las primeras filas en la consola

16. *head(dataset)*

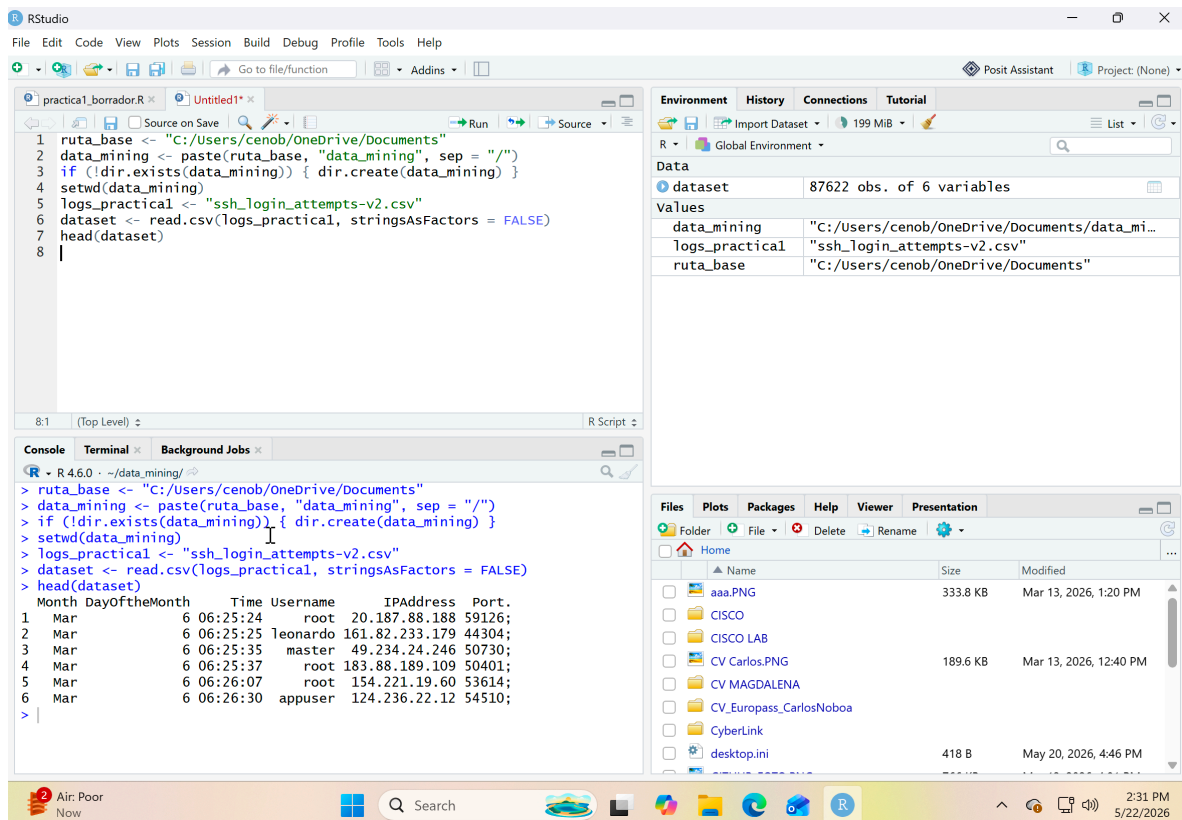


Imagen 1. Resultados en R para gestión de directorios y carga local del CSV

D. Explicación de los comandos usados y su función

Tabla 1. Comandos para gestión de directorios y carga local del CSV

Función / Comando	Descripción Técnica y Operativa	Aplicación Específica en el Script
<code>paste(..., sep = "/")</code>	Concatena (une) cadenas de texto independientes utilizando un carácter de separación especificado.	Toma la dirección general de documentos del sistema (<code>ruta_base</code>) y le anexa la palabra "data_mining" mediante una barra diagonal para construir una ruta de acceso válida.

dir.exists()	Realiza una comprobación lógica en el almacenamiento físico para verificar si existe un directorio. Retorna TRUE si existe o FALSE si no lo encuentra.	Al anteponer el operador de negación (!), invierte el resultado booleano original para que la estructura condicional se ejecute bajo la premisa: "Si NO existe la carpeta".
dir.create()	Interactúa de forma directa con el sistema operativo de la máquina para fabricar físicamente un nuevo directorio en la ubicación exacta asignada.	Crea la carpeta física "data_mining" en el almacenamiento local únicamente si la condición lógica previa determinó que el directorio aún no existía.
setwd() (Set Working Directory)	Modifica y configura el entorno global de la sesión activa en RStudio, designando un directorio específico como la ruta de trabajo predeterminada.	Establece la carpeta verificada como el "cuartel general" del script, permitiendo que R busque o guarde archivos de manera relativa sin necesidad de escribir rutas absolutas complejas.
read.csv()	Procesa, analiza e importa archivos de texto plano estructurados bajo el estándar de valores separados por comas (.csv), mapeando filas y columnas.	Carga los datos del archivo de logs de seguridad hacia la memoria del entorno, transformándolos de forma automática en un objeto bidimensional interactivo de tipo Data.Frame.
head()	Función de diagnóstico preliminar que extrae y despliega en la consola únicamente las seis (6) primeras líneas de registros de un objeto.	Permite validar rápidamente si los encabezados de las columnas y el formato general de las filas de logs se han importado de manera correcta sin saturar la pantalla con miles de renglones.

PREGUNTA 2: Creación de variables y comprobación de clases/tipos de datos

A. Contexto y Planteamiento del Problema

En R, a diferencia de otros lenguajes de programación tradicionales, no se requiere la declaración explícita del tipo de datos al inicializar una variable. El motor del lenguaje infiere y asigna la clase del objeto de manera dinámica según el valor asignado.

En esta sección se crean cuatro elementos de almacenamiento fundamentales para el control de la práctica: el nombre del auditor en formato de texto, un umbral de control numérico entero, una bandera lógica de verificación y un vector unidimensional que extrae y aísla la columna completa de direcciones IP del dataset. Tras la asignación, es una buena práctica de ingeniería de datos interrogar al sistema para validar el tipo de dato abstracto asignado en memoria, garantizando que las futuras funciones operen sin conflictos estructurales.

B. Proceso de desarrollo de la solución

Paso 1. Se define una variable con el nombre del analista que va a realizar la práctica y se comprueba su tipo de dato.

Código: `analista <- "Carlos Noboa"; class(analista)`

Paso 2. Se define una variable que almacenará un número de registros máximos (por ejemplo 50) y se comprueba su tipo de dato.

Código: `registros_maximos <- 50; class(registros_maximos)`

Paso 3. Se define una variable de tipo booleana con el valor verdadero y se comprueba su tipo de dato.

Código: `variable_booleana <- TRUE; class(variable_booleana)`

Paso 4. Se crea un vector que extrae todas las direcciones IP de la columna "IPAddress" del dataset y se comprueba su tipo de dato.

Código: `vector_ips <- dataset$IPAddress; class(vector_ips)`

C. Código en Rstudio

1. # 1. Variable con el nombre del analista que va a realizar la práctica
2. `analista <- "Carlos Noboa"`
3. `class(analista)`

4. # 2. Variable que almacenará un número de registros máximos (por ejemplo 50)
5. `registros_maximos <- 50`
6. `class(registros_maximos)`

7. # 3. Variable booleana
8. `variable_booleana <- TRUE`
9. `class(variable_booleana)`

10. # 4. Un vector con todas las IP de la columna "IPAddress"
11. `vector_ips <- dataset$IPAddress`
12. `class(vector_ips)`

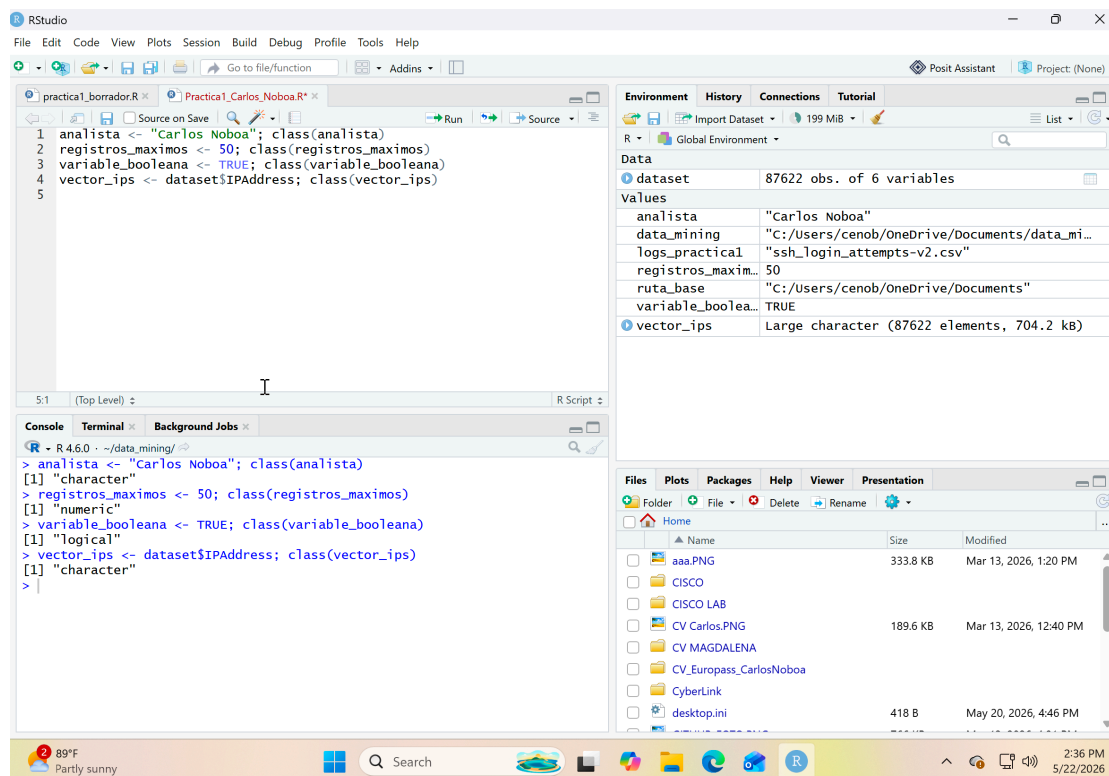


Imagen 2. Resultados en R para creación de variables y comprobación de clases/tipos de datos

D. Explicación de los comandos usados y su función

Tabla 2. Resultados en R para creación de variables y comprobación de clases/tipos de datos

Función / Operador	Descripción Técnica y Operativa	Aplicación Específica en el Script
<- (Asignación)	Símbolo estándar y nativo en el entorno R para almacenar valores dentro de una variable. Inicializa el objeto en el entorno global de trabajo (<i>Global Environment</i>).	Se utiliza de forma sistemática para guardar las cadenas de texto, números enteros, banderas booleanas y vectores bajo identificadores específicos a la izquierda.
\$ (Extracción por Nombre)	Operador de indexación bidimensional. Permite acceder, aislar y extraer componentes o columnas específicas que se encuentran dentro de un DataFrame.	Extrae verticalmente todos los datos pertenecientes a la columna de direcciones de red (IPAddress) del objeto dataset para simplificarlos en un nuevo vector unidimensional.
class()	Comando de diagnóstico, inspección preliminar y metadatos. Interroga al intérprete de R y devuelve una cadena de texto representativa del tipo de dato abstracto de la variable.	Examina de forma secuencial cada una de las variables creadas para validar que almacenen correctamente tipos de datos "character", "numeric" o "logical".

PREGUNTA 3: Operaciones estadísticas con el archivo .csv (Análisis de Logs)

A. Contexto y Planteamiento del Problema

Una vez completada la importación del dataset de auditoría de red, se entra en la fase de minería exploratoria para extraer métricas descriptivas clave. En el contexto de la ciberseguridad, identificar el volumen neto de atacantes únicos e identidades comprometidas es el primer paso para dimensionar una amenaza. Asimismo, estructurar un ranquin jerárquico de incidentes por dirección IP permite mitigar de forma prioritaria los ataques de fuerza bruta activos.

Para que los resultados sean explotables en un informe de ingeniería, el vector plano de frecuencias de R se transforma en una estructura matricial bidimensional ordenada de forma descendente. Además, implementamos una reindexación avanzada: convertimos los nombres de fila desordenados en una columna física estable (ID) y reiniciamos el conteo vertical izquierdo para garantizar la máxima legibilidad de la tabla final.

B. Proceso de desarrollo de la solución

Paso 1. Se calcula la cantidad de direcciones IP únicas registradas en el log.

Código: `ips_unicas <- length(unique(dataset$IPAddress))`

Paso 2. Se muestra en la consola el número total de IPs únicas obtenidas.

Código: `print(ips_unicas)`

Paso 3. Se calcula la cantidad de usuarios únicos registrados en el log.

Código: `usuarios_unicos <- length(unique(dataset$Username))`

Paso 4. Se muestra en la consola el número total de usuarios únicos obtenidos.

Código: `print(usuarios_unicos)`

Paso 5. Se crea una tabla de frecuencias para contar cuántas veces aparece cada dirección IP en el dataset.

Código: `intentos_por_ip <- table(dataset$IPAddress)`

Paso 6. Se convierte la tabla de frecuencias en un formato de objeto data frame.

Código: `tabla_ordenada <- as.data.frame(intentos_por_ip)`

Paso 7. Se asignan nombres específicos a las columnas del nuevo data frame: "IP_Address" e "Intentos".

Código: `colnames(tabla_ordenada) <- c("IP_Address", "Intentos")`

Paso 8. Se ordena el data frame de forma descendente en función de la columna de intentos.

Código: `tabla_ordenada <- tabla_ordenada[order(-tabla_ordenada$Intentos),
]`

Paso 9. Se transforman los nombres de las filas en una columna real llamada "ID" y se une al data frame.

Código: `tabla_ordenada <- cbind(ID = rownames(tabla_ordenada),
tabla_ordenada)`

Paso 10. Se eliminan y reinician los índices repetidos del borde izquierdo asignándoles un valor nulo.

Código: `rownames(tabla_ordenada) <- NULL`

Paso 11. Se imprime en la consola el data frame final completamente estructurado y ordenado.

Código: `print(tabla_ordenada)`

C. Código en Rstudio

1. # 1. Muestra cuántas ip's únicas ha registrado el log.
2. `ips_unicas <- length(unique(dataset$IPAddress))`
3. `print(ips_unicas)`

- 4.
5. # 2. Muestra cuántos usuarios únicos ha registrado el log.
6. `usuarios_unicos <- length(unique(dataset$Username))`
7. `print(usuarios_unicos)`
- 8.
9. # 3. Muestra cuántas veces una ip ha intentado acceso (Formato Vertical Ordenado)
10. `intentos_por_ip <- table(dataset$IPAddress)`
11. `tabla_ordenada <- as.data.frame(intentos_por_ip)`
12. `colnames(tabla_ordenada) <- c("IP_Address", "Intentos")`
13. `tabla_ordenada <- tabla_ordenada[order(-tabla_ordenada$Intentos), ]`
- 14.
15. # Pasamos los números de la izquierda a una columna real llamada "ID"
16. `tabla_ordenada <- cbind(ID = rownames(tabla_ordenada), tabla_ordenada)`
- 17.
18. # Quitamos los números repetidos del borde izquierdo reiniciándolos
19. `rownames(tabla_ordenada) <- NULL`
20. `print(tabla_ordenada)`

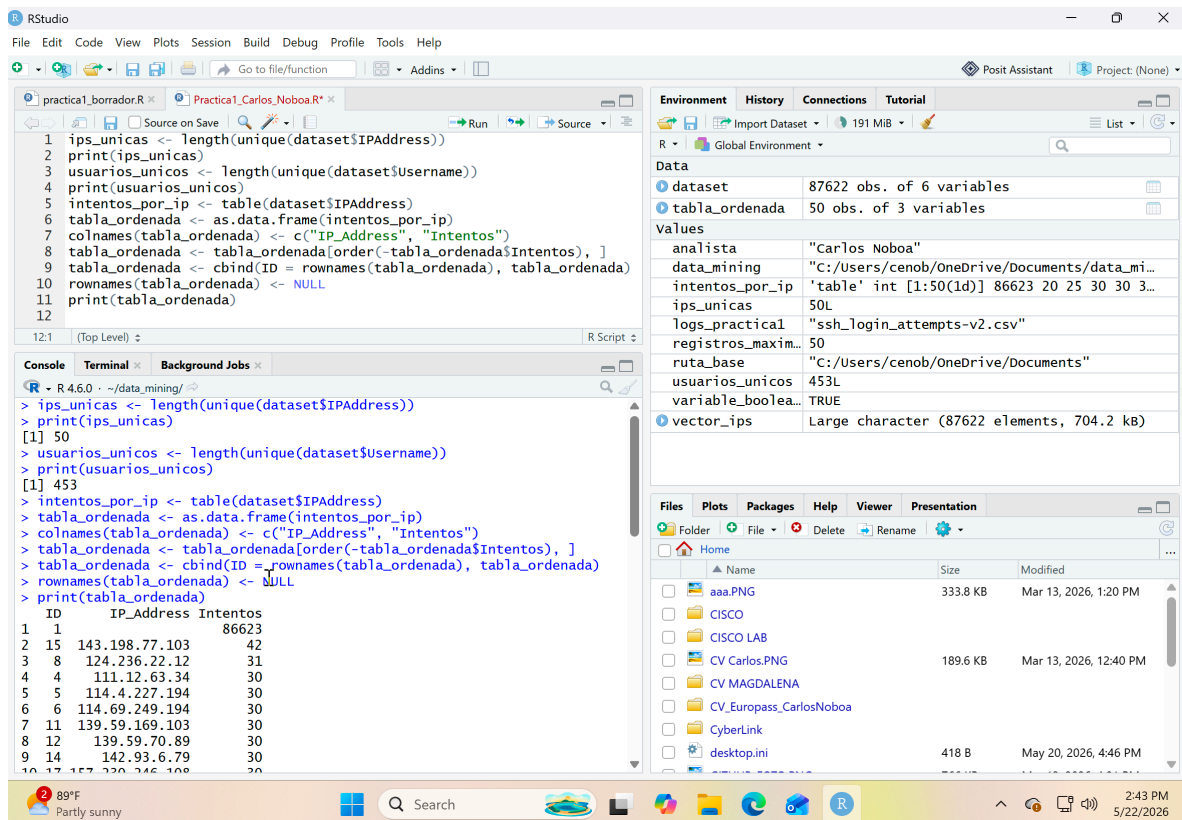


Imagen 3. Resultados en R para operaciones estadísticas con el archivo .csv (Análisis de Logs)

D. Explicación de los comandos usados y su función

Tabla 3. Comandos para operaciones estadísticas con el archivo .csv (Análisis de Logs)

Función / Operador	Descripción Técnica y Operativa	Aplicación Específica en el Script
--------------------	---------------------------------	------------------------------------

unique()	Examina secuencialmente una columna o vector de datos y descarta todos los registros repetidos, devolviendo una colección limpia de valores únicos.	Se aplica sobre las columnas \$IPAddress y \$Username para aislar las direcciones de red y los nombres de usuario distintos que interactuaron con el servidor.
length()	Calcula y reporta el número total de elementos o componentes numéricos que integran una estructura unidimensional.	Permite cuantificar la cantidad exacta de elementos devueltos por unique(), arrojando el conteo neto de IPs y usuarios únicos.
table()	Construye una tabla de contingencia que computa de forma automatizada las frecuencias absolutas de los datos mapeados.	Cuenta cuántas veces exactas aparece repetida cada dirección IP a lo largo de todo el archivo de registros SSH.
as.data.frame()	Convierte colecciones, matrices u objetos tabulares nativos de R en la estructura estándar de un DataFrame .	Transforma el resumen de frecuencias de las IPs en una estructura bidimensional clásica organizada en filas y columnas manipulables.
colnames()	Instrucción de asignación utilizada para consultar, definir o renombrar las cabeceras de las columnas de una tabla.	Reemplaza las etiquetas automáticas de R por los nombres personalizados y descriptivos "IP_Address" e "Intentos".
order()	Evalúa un vector y devuelve permutaciones de sus posiciones para reorganizarlo. El signo menos (-) fuerza una ordenación descendente .	Reordena las filas de la tabla de mayor a menor tomando como eje el volumen de accesos detectados para priorizar las IPs más activas.
rownames()	Accede directamente a los atributos de identificación o nombres asignados a las filas en un DataFrame.	Se usa primero para extraer los índices originales desordenados y, al final, se iguala a NULL para reiniciar el conteo vertical consecutivo desde 1.
cbind() (Column Bind)	Enlaza u acopla vectores y columnas de forma horizontal para expandir estructuralmente un DataFrame existente.	Inserta horizontalmente los antiguos nombres de fila al inicio de la tabla bajo la creación de una columna física formal denominada "ID".
print()	Ordena al intérprete de R forzar la salida estándar de datos para desplegar en la pantalla de la consola el resultado estructurado de una variable.	Muestra en el entorno los resultados numéricos finales y la tabla jerárquica ya formateada de las direcciones IP.

PREGUNTA 4: Operaciones y Consultas sobre un DataFrame

A. Contexto y Planteamiento del Problema

Los DataFrames constituyen la estructura de datos bidimensional más utilizada en R para el análisis de datos masivos, emulando el formato de tablas relacionales con filas (registros) y columnas (variables). En esta sección, se inicializa una tabla de control de usuarios con atributos de nombres, edades y ciudades de residencia.

A partir de esta estructura, se abordan tres tareas analíticas esenciales: la inspección de metadatos mediante la validación de tipos de datos por columna, la extracción selectiva de registros (filtrado por umbral de edad) y el cálculo de estadísticas de tendencia central (media aritmética) bajo criterios lógicos multivariable. Estas operaciones simulan las fases de segmentación y limpieza de perfiles habituales en los procesos de Data Mining.

B. Proceso de desarrollo de la solución

Paso 1. Se define un data frame de prueba llamado `df` con los nombres, edades y ciudades provistos por el enunciado.

Código:

```
df <- data.frame(Nombre = c("Pepe", "Juan", "Maria", "Antonia"),
  Edad = c(15, 35, 25, 41), Ciudad = c("Barcelona", "Castello", "Girona",
  "Alacant"))
```

Paso 2. Se aplica la función `class` a cada columna del data frame para identificar sus tipos de datos.

Código: `clases_columnas <- sapply(df, class)`

Paso 3. Se muestra en la consola el tipo de dato correspondiente a cada columna.

Código: `print(clases_columnas)`

Paso 4. Se realiza un filtrado en el data frame para extraer las filas de las personas que tienen una edad mayor a 27 años.

Código: `personas_mayores_27 <- df[df$Edad > 27, ]`

Paso 5. Se muestra en la consola el subconjunto de datos con las personas mayores de 27 años.

Código: `print(personas_mayores_27)`

Paso 6. Se filtran las filas del data frame para seleccionar únicamente a las personas que viven en "Castello" o en "Alacant".

Código: `filtro_diez_ciudades <- df[df$Ciudad == "Castello" | df$Ciudad == "Alacant", ]`

Paso 7. Se calcula el promedio numérico de la columna de edad basándose en el subconjunto de ciudades filtrado.

Código: `media_edad <- mean(filtro_diez_ciudades$Edad)`

Paso 8. Se muestra en la consola el resultado del cálculo de la media de edad obtenido.

Código: `print(media_edad)`

C. Código en Rstudio

```
1. # Definición del Data.Frame provisto por el enunciado
2. df <- data.frame(
3.   Nombre = c("Pepe", "Juan", "Maria", "Antonia"),
4.   Edad = c(15, 35, 25, 41),
5.   Ciudad = c("Barcelona", "Castello", "Girona", "Alacant")
6. )
7.
8. # 1. Ver la clase de cada columna de nuestra tabla de personas
9. clases_columnas <- sapply(df, class)
10. print(clases_columnas)
11.
```

12. # 2. Ver las personas con edad mayor a 27 años
 13. `personas_mayores_27 <- df[df$Edad > 27, ]`
 14. `print(personas_mayores_27)`
 15.
 16. # 3. Calcular la media de edad de las personas que son de Castelló y Alacant
 17. `filtro_ciudades <- df[df$Ciudad == "Castello" | df$Ciudad == "Alacant", ]`
 18. `media_edad <- mean(filtro_ciudades$Edad)`
 19. `print(media_edad)`

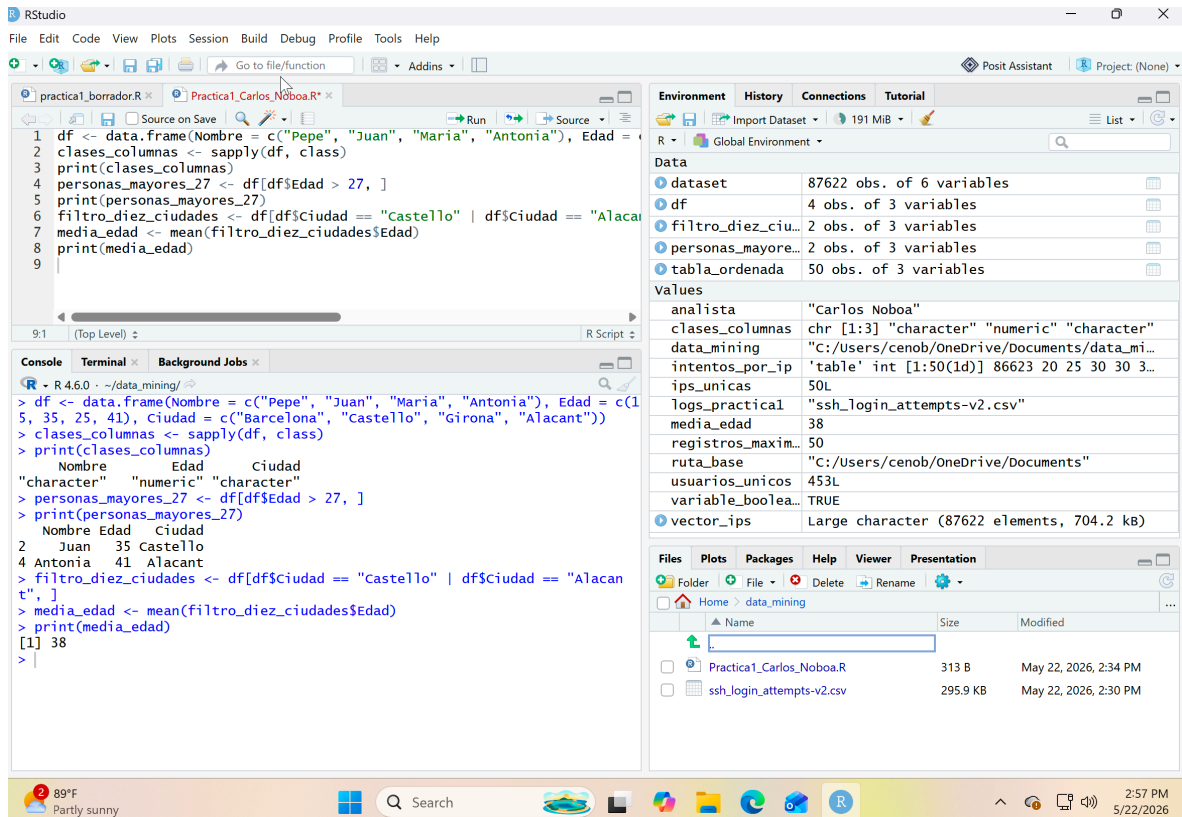


Imagen 4. Resultados en R para Operaciones y Consultas sobre un Data.Frame

D. Explicación de los comandos usados y su función

Tabla 4. Comandos para operaciones y Consultas sobre un Data.Frame

Función / Operador	Descripción Técnica y Operativa	Aplicación Específica en el Script
<code>data.frame()</code>	Estructura constructora encargada de generar colecciones de datos bidimensionales en R, emulando el formato clásico de una tabla de bases de datos relacionales (filas y columnas).	Se utiliza para inicializar la tabla de usuarios llamada <code>df</code> , agrupando de forma alineada los vectores de nombres, edades y ciudades de residencia.
<code>sapply()</code>	Función vectorizada de la familia <i>apply</i> . Recorre de manera optimizada cada componente o columna de una estructura para aplicarle una directiva común de forma simultánea.	Examina de golpe las tres columnas del <code>DataFrame</code> (<code>df</code>) y ejecuta sobre ellas la función <code>class</code> para extraer de manera limpia sus respectivos tipos de datos.
<code>class()</code>	Comando de diagnóstico de metadatos. Interroga al motor de R para conocer la naturaleza abstracta y el formato de almacenamiento de una estructura o variable.	Determina que las columnas de texto se almacenen bajo la clase "character" y la columna de edad bajo la clase "numeric".

[,] (Corchetes de Indexación)	Operador de filtrado dimensional basado en la sintaxis de coordenadas [filas, columnas]. Al colocar una condición lógica antes de la coma, se aíslan registros específicos.	Permite filtrar y extraer filas completas del DataFrame; en el primer caso rescata registros según el umbral Edad > 27, y en el segundo según la coincidencia de localidades.
== (Comparación Lógica)	Operador de igualdad estricta. Evalúa elemento por elemento si los valores de un vector coinciden exactamente con la cadena de texto o número indicado.	Verifica qué celdas de la columna \$Ciudad corresponden de manera exacta a los términos "Castello" o "Alacant".
 (Operador Lógico OR / O)	Enlaza múltiples condiciones lógicas evaluando si al menos una de ellas se cumple (es verdadera) para dar paso al rescate de la fila correspondiente.	Permite que el filtro extraiga el registro completo si la ciudad del usuario es Castelló O si es Alacant de forma inclusiva.
mean()	Función estadística de tendencia central que calcula de manera directa la media aritmética (promedio) de un vector o conjunto numérico.	Suma las edades de las personas que pasaron el filtro de las dos ciudades seleccionadas y las divide de forma automática por el número total de coincidencias.
print()	Ordena al intérprete forzar la salida estándar para imprimir en la pantalla de la consola el contenido estructurado o resultado numérico de una variable.	Despliega en el entorno de desarrollo el reporte de las clases, las tablas filtradas y el valor numérico final del promedio de edad.

PREGUNTA 5: Operaciones Avanzadas con Listas

A. Contexto y Planteamiento del Problema

Las listas son las estructuras de datos más flexibles y generales en R, ya que actúan como contenedores heterogéneos capaces de almacenar colecciones de objetos con diferentes formas, longitudes y naturalezas (vectores numéricos, cadenas de texto, matrices o valores lógicos) bajo un único identificador jerárquico.

En esta sección se analiza una lista compuesta por una secuencia matemática incremental (A), un subvector de caracteres del abecedario (B), una cadena de texto aislada (C) y un vector de máscaras lógicas basadas en residuos aritméticos (D). A partir de este objeto, se resuelven tres requerimientos analíticos avanzados: el diagnóstico automatizado de clases por nodo, la mutación condicional para transformar texto plano en factores categóricos estructurados sin alterar el resto de componentes, y la indexación posicional cruzada para extraer subconjuntos numéricos específicos.

B. Proceso de desarrollo de la solución

Paso 1. Se define una lista llamada `l1` que contiene cuatro elementos (A, B, C y D) formados por una secuencia numérica, letras del abecedario, un carácter de texto y una condición lógica.

Código: `l1 <- list(A = seq(from = 10, to = 122, by = 4), B = letters[3:13], C = c("Juanito"), D = (1:30 %% 5 == 0 | 1:30 %% 3 == 0))`

Paso 2. Se aplica la función `class` a cada elemento de la lista para identificar a qué tipo de dato pertenece.

Código: `clases_lista <- lapply(l1, class)`

Paso 3. Se muestra en la consola el tipo de dato correspondiente a cada elemento de la lista.

Código: `print(clases_lista)`

Paso 4. Se aplica una función personalizada a la lista para buscar los elementos de tipo carácter y transformarlos en factores, guardando el resultado en una nueva lista modificada.

Código: `l1_modificada <- lapply(l1, function(x) { if (is.character(x)) {
return(as.factor(x)) } else { return(x) } })`

Paso 5. Se muestra en la consola la estructura de la lista modificada con los nuevos elementos de tipo factor.

Código: `print(l1_modificada)`

Paso 6. Se realiza un filtrado del elemento "A" de la lista utilizando las posiciones verdaderas establecidas por el vector lógico del elemento "D".

Código: `valores_filtrados <- l1$A[l1$D]`

Paso 7. Se muestra en la consola el resultado de los valores filtrados obtenidos.

Código: `print(valores_filtrados)`

C. Código en Rstudio

```
1. # Definición de la lista provista por el enunciado
2. l1 <- list(
3.   A = seq(from = 10, to = 122, by = 4),
4.   B = letters[3:13],
5.   C = c("Juanito"),
6.   D = (1:30 %% 5 == 0 | 1:30 %% 3 == 0)
7. )
8.
9. # Encontrar de qué clase es cada elemento de la lista
10. clases_lista <- lapply(l1, class)
11. print(clases_lista)
12.
13. # Convertir elementos de tipo carácter a factor y devolver la lista modificada
14. l1_modificada <- lapply(l1, function(x) {
15.   if (is.character(x)) {
16.     return(as.factor(x))
17.   } else {
18.     return(x)
19.   }
20. })
21. print(l1_modificada)
22.
```

23. # Seleccionar aquellos valores de “A” según el vector lógico “D” (donde sea TRUE)
 24. *valores_filtrados <- l1\$A[l1\$D]*
 25. *print(valores_filtrados)*

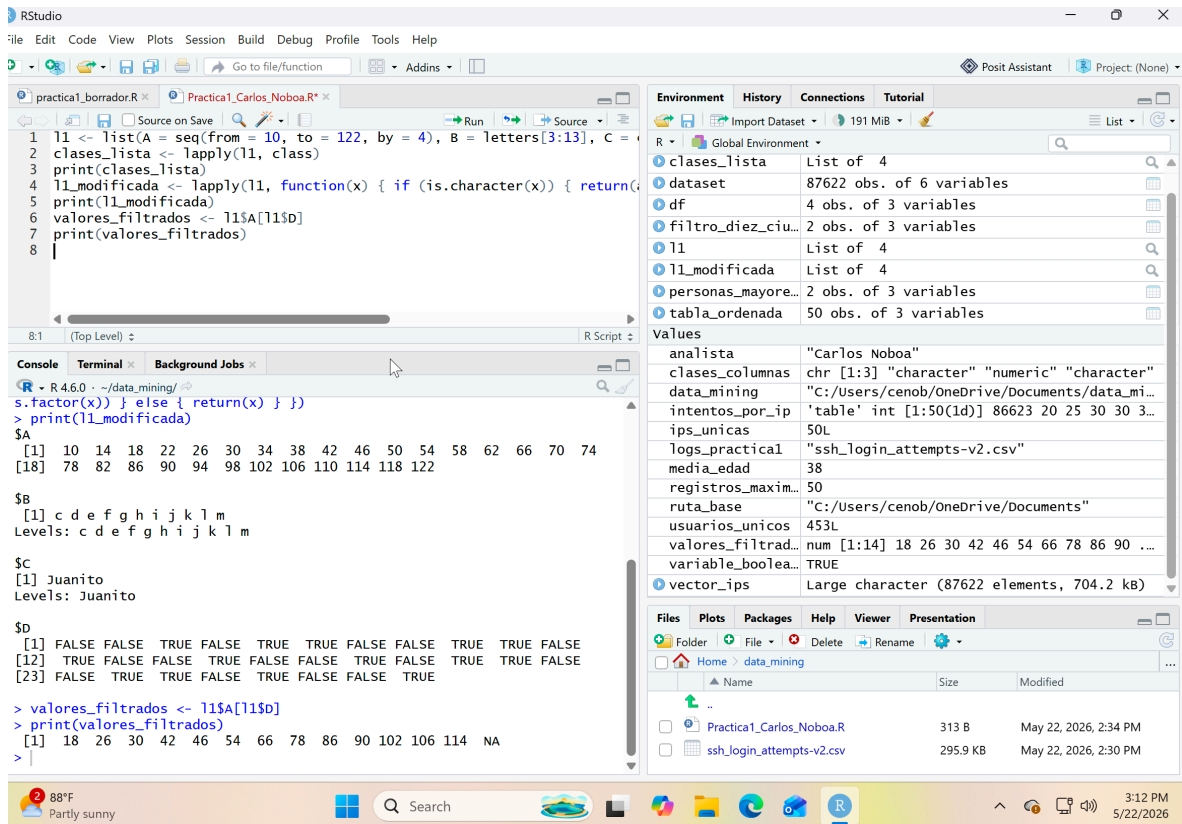


Imagen 5. Resultados en R para Operaciones Avanzadas con Listas

D. Explicación de los comandos usados y su función

Tabla 5. Comandos para Operaciones Avanzadas con Listas

Función / Operador	Descripción Técnica y Operativa	Aplicación Específica en el Script
list()	Estructura constructora jerárquica y heterogénea en R. Es capaz de agrupar y almacenar colecciones de objetos independientes (vectores, cadenas, matrices) con diferentes longitudes y naturalezas bajo un único identificador.	Se utiliza para inicializar la lista objeto l1, organizando secuencialmente un vector numérico incremental (A), una secuencia de letras (B), un texto aislado (C) y un vector de máscaras lógicas (D).
seq()	Función generadora de secuencias matemáticas continuas o discretas a partir de un valor inicial (from), un límite superior (to) y un intervalo de salto constante (by).	Genera la secuencia numérica que se almacena en el compartimento \$A, comenzando en 10 y finalizando de forma exacta en 122 con incrementos de 4 en 4.
letters[]	Vector nativo integrado en R que contiene las 26 letras del abecedario en minúscula. El operador de corchetes permite extraer posiciones indexadas específicas.	Aísla un subvector de letras que va desde la tercera posición hasta la decimotercera (3:13), correspondiente al rango alfabético desde la "c" hasta la "m".
%%	Operador aritmético que calcula de forma directa el resto o residuo numérico resultante de una división entera entre dos elementos.	Evalúa la serie numérica del 1 al 30 de forma paralela para comprobar qué posiciones

(Módulo / Residuo)		devuelven residuo igual a cero al dividirse entre 5 o entre 3.
lapply() (List Apply)	Función de alto rendimiento perteneciente a la familia <i>apply</i> . Recorre de forma secuencial cada nodo de una lista, ejecuta una directiva y devuelve de manera obligatoria una nueva lista con las respuestas empaquetadas.	En el primer caso, recorre l1 para extraer las clases de datos con class. En el segundo, mapea la lista completa a través de una función personalizada condicional.
class()	Comando analítico de metadatos. Interroga al intérprete para conocer el tipo de almacenamiento abstracto de una estructura o variable.	Diagnostica y reporta la naturaleza de los compartimentos en el orden "numeric", "character", "character" y "logical".
function(x) { ... } (Función Anónima)	Bloque lógico personalizado que se declara temporalmente "al vuelo" dentro de un operador para procesar tareas a medida sin necesidad de registrar un comando permanente en el entorno global.	Define la lógica condicional que evalúa cada nodo de la lista para discriminar si su contenido corresponde o no a una cadena de texto.
is.character()	Evaluator lógico de tipado. Comprueba un objeto y retorna una bandera booleana (TRUE o FALSE) para indicar si la variable evaluada almacena texto plano.	Actúa como el gatillo del condicional if para detectar que únicamente los compartimentos \$B y \$C deben ingresar a la fase de conversión.
as.factor()	Comando de conversión cualitativa que transforma texto plano o numérico en factores , una estructura estadística optimizada que mapea categorías mediante niveles jerárquicos (<i>Level/s</i>).	Transforma las cadenas de texto plano de la lista en variables categóricas cualitativas estructuradas con sus respectivos niveles.
\$ y [] (Indexación Cruzada)	Combinación de operadores. El símbolo \$ extrae componentes individuales por su etiqueta y los corchetes [] aplican filtros de selección posicional (máscaras lógicas).	Extrae los elementos numéricos de l1\$A utilizando como filtro de rescate las posiciones donde el vector booleano l1\$D es estrictas y explícitamente verdadero (TRUE).
print()	Ordena al intérprete forzar la salida estándar de datos para desplegar en la pantalla de la consola el resultado estructurado o contenido de una variable.	Imprime en la consola de RStudio los listados de metadatos de las clases, la estructura de la lista mutada y el vector de valores numéricos filtrados.

PREGUNTA 6: Funciones Personalizadas y Vectorización

A. Contexto y Planteamiento del Problema

Una de las mayores ventajas de R en el Data Mining y la analítica avanzada es su capacidad nativa para trabajar de forma **vectorizada**. A diferencia de los lenguajes tradicionales que requieren bucles iterativos lentos (como `for` o `while`) para recorrer un conjunto de datos elemento por elemento, R puede realizar cálculos matemáticos complejos sobre bloques completos de memoria en una sola operación de CPU.

En esta sección, se define inicialmente un vector numérico incremental (`v1`) mediante una secuencia de saltos constantes. A partir de este objeto, se construyen dos funciones lógicas modulares y reutilizables: la primera calcula la suma de aquellos elementos situados exclusivamente en posiciones indexadas pares mediante submapeos algebraicos, y la segunda actúa como un módulo resumen de metadatos analíticos para reportar los límites máximo, mínimo y el volumen total de observaciones utilizando funciones nativas de alto rendimiento.

B. Proceso de desarrollo de la solución

Paso 1. Se define un vector numérico de prueba llamado `v1` compuesto por una secuencia del 5 al 15 con incrementos de 2 en 2.

Código: `v1 <- seq(from = 5, to = 15, by = 2)`

Paso 2. Se declara una función personalizada llamada `suma_posiciones_pares` que recibe un vector numérico, genera una secuencia de índices del 1 hasta la longitud total del mismo, y filtra los elementos que se encuentran únicamente en las posiciones pares (donde el residuo de la división por 2 es 0) para sumarlos y retornar el resultado.

Código: `suma_posiciones_pares <- function(vector_numeric) { indices <- 1:length(vector_numeric); resultado_suma <- sum(vector_numeric[indices %% 2 == 0]); return(resultado_suma) }`

Paso 3. Se ejecuta la función `suma_posiciones_pares` utilizando como argumento el vector `v1` y se almacena el valor resultante en una variable.

Código: `resultado_q6_1 <- suma_posiciones_pares(v1)`

Paso 4. Se muestra en la consola el resultado del cálculo de la suma de las posiciones pares.

Código: `print(resultado_q6_1)`

Paso 5. Se declara una función personalizada llamada `analizar_vector` que toma un vector de números y construye una lista interna con el valor máximo, el valor mínimo y la longitud total del mismo, retornando dicha lista al finalizar.

Código: `analizar_vector <- function(vector_numeric) { lista_resultados <- list(Maximo = max(vector_numeric), Minimo = min(vector_numeric), Longitud = length(vector_numeric)); return(lista_resultados) }`

Paso 6. Se ejecuta la función `analizar_vector` pasando el vector `v1` como parámetro y se guarda la lista generada en una nueva variable.

Código: `resultado_q6_2 <- analizar_vector(v1)`

Paso 7. Se muestra en la consola la lista final con las métricas del vector analizado (máximo, mínimo y longitud).

Código: `print(resultado_q6_2)`

C. Código en Rstudio

1. # Definición del vector provisto por el enunciado
2. `v1 <- seq(from = 5, to = 15, by = 2)`
3. # 1. Función con vectorización para sumar elementos en posiciones pares
4. `suma_posiciones_pares <- function(vector_numeric) {`
5. # Generamos una secuencia de índices del 1 a la longitud del vector
6. `indices <- 1:length(vector_numeric)`

```

7. # Filtramos el vector donde las posiciones sean pares y sumamos el bloque
8. resultado_suma <- sum(vector_numerico[indices %% 2 == 0])
9. return(resultado_suma)
10. }
11. # Aplicamos la función sobre nuestro vector v1 como ejemplo
12. resultado_q6_1 <- suma_posiciones_pares(v1)
13. print(resultado_q6_1)
14. # 2. Función nativa para reportar Máximo, Mínimo y Longitud
15. analizar_vector <- function(vector_numerico) {
16. lista_resultados <- list(
17. Maximo = max(vector_numerico),
18. Minimo = min(vector_numerico),
19. Longitud = length(vector_numerico)
20. )
21. return(lista_resultados)
22. }
23. # Aplicamos la función sobre nuestro vector v1 como ejemplo
24. resultado_q6_2 <- analizar_vector(v1)
25. print(resultado_q6_2)

```

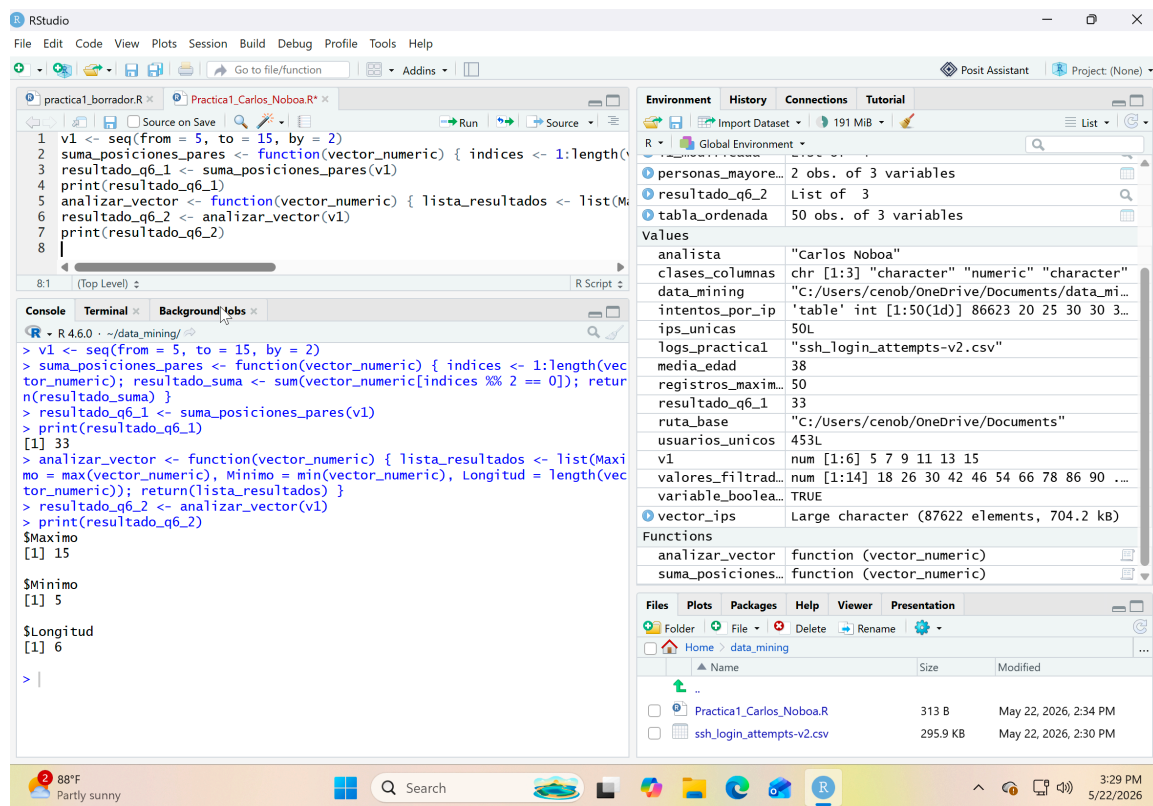


Imagen 6. Resultados en R para Funciones Personalizadas y Vectorización

D. Explicación de los comandos usados y su función

Tabla 6. Comandos para Funciones Personalizadas y Vectorización

Función / Operador	Descripción Técnica y Operativa	Aplicación Específica en el Script
seq()	Función generadora de secuencias matemáticas continuas o discretas a partir de un valor inicial (from), un límite superior (to) y un intervalo de salto constante (by).	Inicializa el vector numérico de prueba v1, construyendo una serie de datos que inicia en 5 y termina en 15 con incrementos constantes de 2 en 2.
function() { ... }	Estructura constructora utilizada para declarar, empaquetar e inicializar funciones personalizadas en R. Permite encapsular bloques lógicos reutilizables asignando parámetros de entrada.	Define las rutinas modulares suma_posiciones_pares (para operaciones aritméticas selectivas) y analizar_vector (para extracción de estadísticas descriptivas).
1:length()	Genera un rango o serie consecutiva de números enteros que va desde la unidad (1) hasta la longitud neta medida en el objeto. Actúa como un constructor dinámico de mapas de índices.	Crea el vector secuencial de posiciones numéricas índices que refleja el orden posicional exacto (del 1 al 6) de los componentes del vector evaluado.
length()	Función analítica que calcula y reporta el número total de elementos o componentes que integran una estructura unidimensional o vector numérico.	Se utiliza en la primera función para delimitar dinámicamente la secuencia de índices y en la segunda función para reportar la longitud total como metadato del vector.
[] (Corchetes de Indexación)	Operador de filtrado y selección posicional. Extrae exclusivamente aquellos componentes del vector original cuyas posiciones coinciden con los criterios lógicos indicados dentro de los corchetes.	Aísla los valores del vector numérico de entrada tomando únicamente aquellos cuyos índices de posición cumplen la condición de ser números pares.
%% (Módulo / Residuo)	Operador aritmético que calcula de forma directa el resto o residuo numérico resultante de una división entera entre dos elementos.	Evalúa el vector de índices mediante la condición índices %% 2 == 0 para discriminar de forma matemática qué posiciones corresponden a índices pares (residuo cero).
== (Comparación Lógica)	Operador de igualdad estricta. Evalúa elemento por elemento si los valores de un objeto numérico o textual coinciden exactamente con la condición asignada.	Valida si el residuo resultante de la operación módulo es igual a cero de forma exacta, devolviendo una máscara lógica de vectores booleanos.
sum()	Función matemática nativa de alta velocidad encargada de calcular la adición total o sumatoria de todos los componentes numéricos pasados como parámetro.	Realiza una suma directa vectorizada sobre el bloque de valores numéricos que fueron previamente aislados por corresponder a las posiciones pares.
return()	Directiva de control de flujo de salida de una función. Detiene inmediatamente la ejecución interna del bloque y devuelve el objeto resultante al entorno principal del sistema.	Envía hacia el entorno global el resultado final de la sumatoria (resultado_suma) o la estructura empaquetada de métricas (lista_resultados).
list()	Estructura constructora jerárquica y heterogénea. Agrupa múltiples variables u objetos independientes conservando etiquetas personalizadas para cada compartimento.	Empaqueta los tres resultados analíticos del vector (Maximo, Minimo y Longitud) dentro de una sola colección estructurada de variables.
max()	Función estadística primitiva de inspección que examina la totalidad de un vector numérico y extrae de manera instantánea el valor más alto (máximo) presente.	Localiza e identifica el elemento numérico de mayor valor dentro de la colección provista en el parámetro (vector_numerico).
min()	Función estadística primitiva de inspección que examina la totalidad de un vector numérico y extrae de manera instantánea el valor más bajo (mínimo) presente.	Localiza e identifica el elemento numérico de menor valor dentro de la colección provista en el parámetro (vector_numerico).
print()	Ordena al intérprete forzar la salida estándar de datos para desplegar en la pantalla de la consola el resultado estructurado o contenido de una variable.	Imprime en la consola de RStudio el valor numérico final de la sumatoria de las posiciones pares y el listado de resultados estadísticos del vector.

CONCLUSIONES

La realización de esta práctica demuestra que la automatización de tareas de bajo nivel y el control dinámico del entorno de trabajo a través de R constituyen pilares fundamentales para garantizar la reproducibilidad en proyectos de Data Mining. Al integrar la verificación de directorios en la nube con la ingesta estructurada del archivo de logs de seguridad, se optimiza la portabilidad del código y se mitigan los errores humanos de configuración. Asimismo, la exploración estadística descriptiva aplicada sobre los registros SSH evidencia cómo el procesamiento masivo de datos crudos permite transformar eventos de red dispersos en una matriz analítica ordenada y priorizada, resultando indispensable para identificar con rapidez vectores de ataque de fuerza bruta en infraestructuras críticas.

Por otra parte, la resolución de las consultas avanzadas sobre Data.Frames, listas y funciones personalizadas pone de manifiesto el potencial de la programación funcional y la vectorización frente a los bucles iterativos tradicionales. El uso estratégico de operadores lógicos de filtrado y herramientas masivas de diagnóstico demuestra que R permite segmentar, transformar y aislar subconjuntos de datos en un solo bloque de memoria sin recurrir a estructuras lentas de cómputo. Este enfoque analítico no solo acelera significativamente los tiempos de respuesta del sistema ante grandes volúmenes de datos, sino que además asienta las bases metodológicas necesarias para el modelado de variables categóricas y la toma de decisiones en ciberseguridad.